

Algorithms for Data Science
Exam 2025
Group C

June 2025

Your Name: _____ Your Student ID: _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. 5 points Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1. Coordinates may be arbitrarily large and all start at height 0. The function `blocks(pos, h)` takes the list `pos` of drop positions and the target height `h`, and returns the x-coordinate where a column's height first reaches `h`.

Hint: the task is related to the third programming task.

```

1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7          # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10             # ??? [Line 10]
11             return x

```

Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

- | | |
|--|--|
| • Line 3: <u> O </u> | • Line 9: <u> C </u> |
| • Line 6: <u> A </u> | |
| • Line 7: <u> I </u> | • Line 10: <u> M </u> |

Code snippets to choose from:

- | | |
|--|---|
| • (A) <code>if x not in heights:</code> | • (I) <code>heights[x] = 1</code> |
| • (B) <code>if x < len(heights):</code> | • (J) <code>if heights[x] == 0:</code> |
| • (C) <code>heights[x] += 1</code> | • (K) <code>heights[i] = x</code> |
| • (D) <code>if heights[x] < h:</code> | • (L) <code>heights = [0] * max(pos)</code> |
| • (E) <code>heights[x] = h</code> | • (M) <code>if heights[x] == h:</code> |
| • (F) <code>heights[x] += i</code> | • (N) <code>heights[i] += x</code> |
| • (G) <code>if heights[h] == i:</code> | • (O) <code>heights = {}</code> |
| • (H) <code>heights = [0] * n</code> | |

Note: The indentation of your code is the same as of `# ???`. In other words, your code for an empty line will start at the position of #.

2. 4 points Complete the following algorithm **AlgoA**($a[1..n]$) such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 17T(\lceil n/30 \rceil) + \Theta(n^8) .$$

You don't need to understand what the algorithm computes!

The input of **AlgoA**($a[1..n]$) is an array a of n integers.

The input of the auxiliary function **Copy_Sub_Array**($a[1..n], i, m$) is an array $a[1..n]$, a position i and a length m . The function returns a copy of the subarray $a[i+1..i+m]$.

<pre> AlgoA($a[1..n]$) if $n < 31$ then return 25 else $d = \underline{\hspace{1cm}5\hspace{1cm}}$ $m = \lceil n / (6 \cdot d) \rceil$ $sum = 0$ for $i = 1$ to 6 do $b = \text{Copy_Sub_Array}(a, i, m)$ $sum = sum + \text{AlgoA}(b)$ for $i = 1$ to <u>11</u> do $b = \text{Copy_Sub_Array}(a, i, m)$ $sum = sum + \text{AlgoA}(b)$ $w = 1$ for $j = 1$ to <u>8</u> do $w = w \cdot n$ // Time: $O(1)$ for $k = 1$ to w do $sum = sum + a[31]$ return sum </pre>	<pre> Copy_Sub_Array($a[1..n], i, m$) $b[1..m] = \text{new empty array}$ for $j = 1$ to m do $b[j] = a[i + j]$ return b </pre>
---	---

Fill in the three blanks in the algorithm above, each time writing a **single appropriate integer number**!

3. 4 points Solve the following recurrence using the master theorem. Determine (for yourself) the values for a , b , and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 100^3 \cdot 100 \cdot T(n/100) + \Theta(n^3)$$

Your answers:

$$c = \underline{\mathbf{4}}$$

Case: **A**

Resulting running time $T(n) = \underline{\hspace{2cm} \Theta(n^4) \hspace{2cm}}$

4. Simplify the following running times as far as possible:

(a) 2 points $\Theta(4n^3 + 12n^2 \log n + 2n^3) = \Theta(\underline{\hspace{2cm} n^3 \hspace{2cm}})$

(b) 2 points $\Theta(2 \cdot n^{(3-3)} + 5 + 2^{(n-n)} \cdot 4) = \Theta(\underline{\hspace{2cm} 1 \hspace{2cm}})$

(c) 2 points $\Theta((10^{\log_{10} n})^{10} + (2^{n/10})^2) = \Theta(\underline{\hspace{2cm} 2^{n/5} \hspace{2cm}})$

5. 5 points Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 100, 40, 95, 90, 25, 30, 65, 80, 50, 85, 20, 15, 30, 35, 55, 60, 75, 70, 50, 45 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?

(Write a single number between 1 and 21; there is only one correct answer.)

Your answer: $j =$ 7 (the violating edge's other endpoint is at 15.)

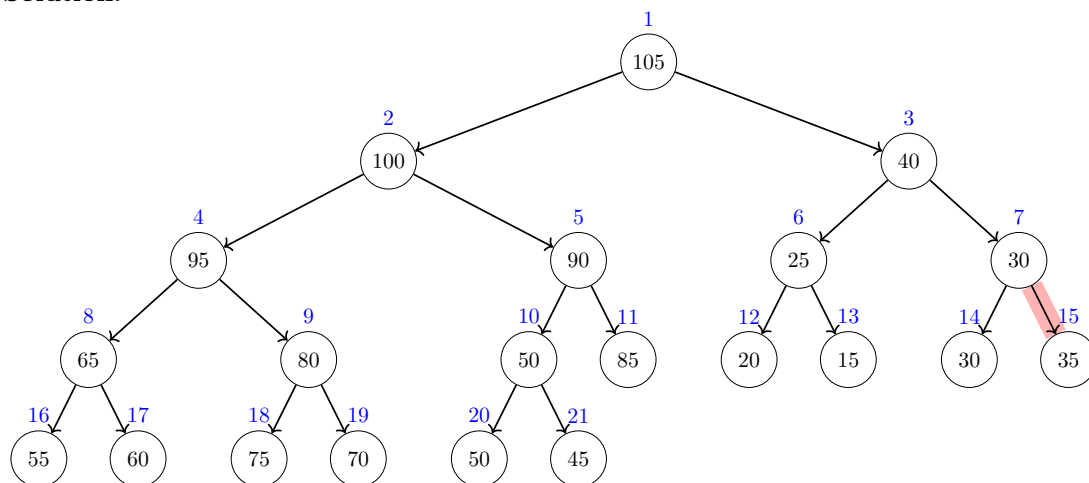
What is the new value for $a[j]$?

(Write a single number, as large as possible and larger than the old value of $a[j]$.)

Your answer: $a[j] =$ 40 (the largest value from $[35, 40]$)

Your tree should contain **all** the elements of the array, from $a[1] = 105$ up to $a[21] = 45$:

Solution:



*Explanation (not required): The heap invariant is violated at the red edge as the parent (index **7**) has a smaller value than its child (index **15**). To fix the heap invariant, it suffices to increase $a[7]$ to any value between 35 and 40. The largest possible value (as asked in the task) is thus 40.*

6. 9 points Dijkstra's algorithm (see below) is run on the following graph where the start node is s .

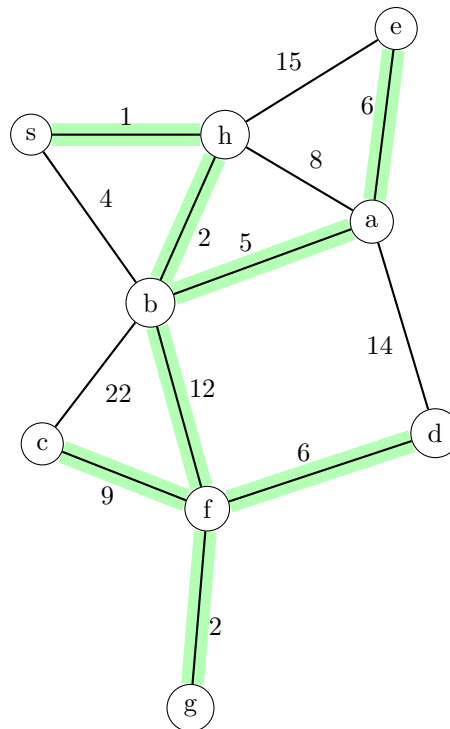
What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```

dist[0..n-1] filled with INF
parent[0..n-1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v,w) do
            if dist[w] > dist[v] + cv,w then
                dist[w] = dist[v] + cv,w
                parent[w] = v
                q.add(w, -dist[w])

```



Solution:

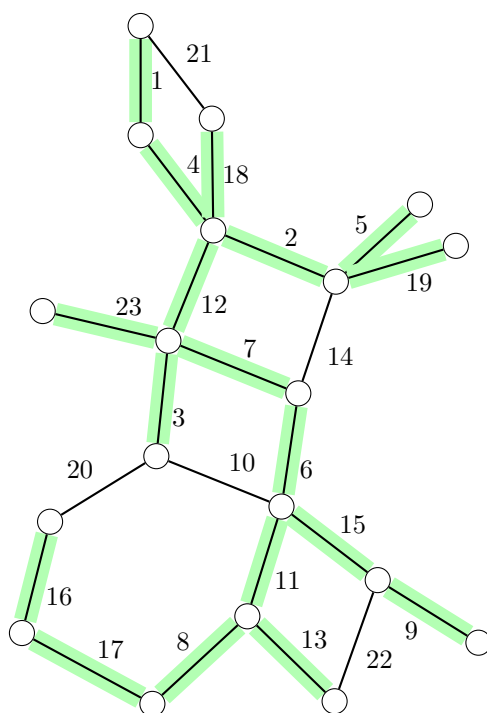
	a	b	c	d	e	f	g	h	s
distance	8	3	24	21	14	15	17	1	0
parent	b	h	f	f	a	b	f	s	NONE

Note:

- The entries of the parent table consist of the names of the nodes or the special value *NONE*. For example, the parent of some node could be s whereas the parent of s is *NONE*.
- If a vertex has more neighbors, we consider them **in lexicographic order** (that is, node a would come before node b and so on)!

7. 5 points Mark all the edges that belong to the minimum spanning tree of the following graph.
(Marking an edge means to draw along that edge.)

Solution:



8. 4 points Add the following keys (in the order as given) into the hash table below using the hash function

$$h(k) = k \bmod 14$$

Keys: 24, 10, 38, 8

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:

Hash table:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
		16						36		52			
		2						22		24			
								8		10			
										38			
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

9. 4 points What is the most appropriate data structure for the following scenario, chosen from the list below?

Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

You are implementing the task scheduling subsystem of a web browser. You record each visited URL; when the user clicks “Back”, you must retrieve and remove the most recently visited URL. Efficiently adding a new URL and undoing to the last URL is critical. URL identifiers can be arbitrarily large integers.

1. Array indexed by keys
2. AVL tree
3. Find-union
4. Hash table
5. Heap/Priority queue
6. Sorted Linked list
7. Queue
8. Stack
9. Sorted array
10. Unsorted array

Most appropriate data structure: Stack

Explanation:

Solution: In this browser-history context, we always need to undo the very last navigation, and never any older one. Thus a last-in-first-out discipline (as realized by a stack) is required to get the correct URL.

The running times of a stack are best possible as **Add** and **ExtractNewest** take only constant time. The memory usage is also best possible as the size of the stack is proportional to the number of elements it contains.

All the other data structures are worse under this scenario as they don't provide feasible operations (queue, heap, find-union), take more time in the worst case (AVL tree, hash table, sorted/unsorted array, sorted linked list), or take too much memory as the maximum key is very large (array indexed by keys).

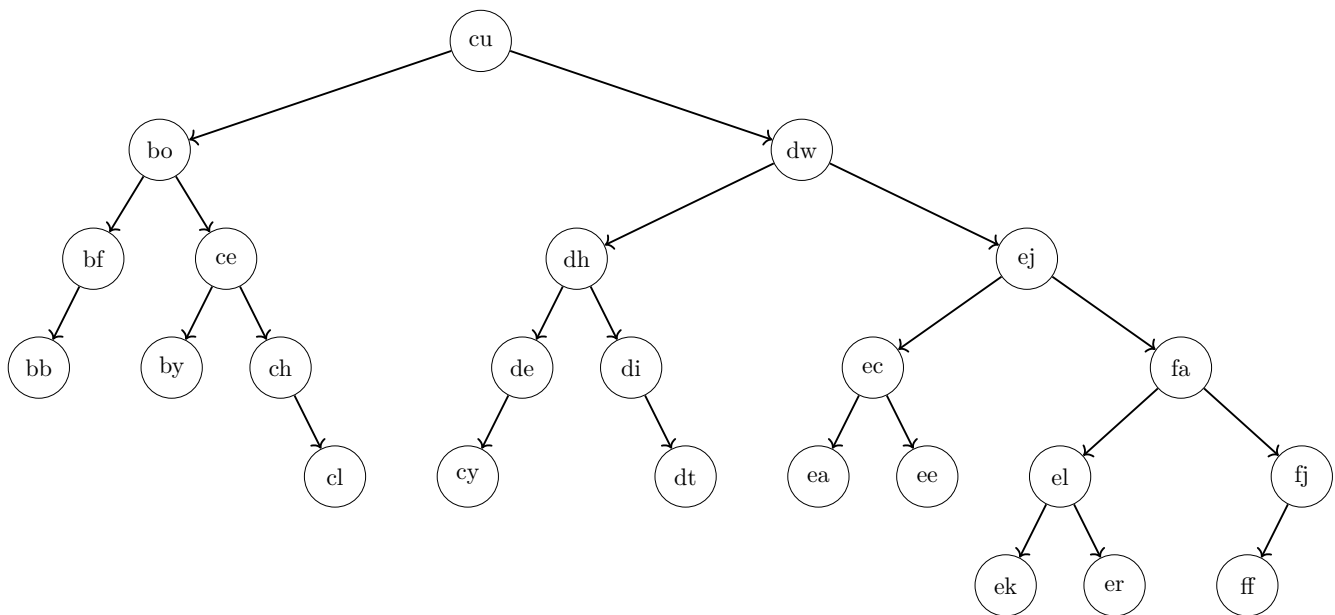
At first glance, an unsorted array may also seem like a good candidate for the best data structure: new elements can be appended to the end in $O(1)$ amortized time, and removing the most recently added element (i.e., the one at the end of the array) can also be done in amortized constant time. For many practical applications, such performance is sufficient. However, in the scenario of this task, minimizing the worst-case time is crucial. Even though we always remove from the end, the worst-case time for both adding and removing elements in an unsorted array is $\Theta(n)$, since a resize may be required which involves allocating a new array and copying all elements. Therefore, the stack outperforms the unsorted array in this setting, as both its add and remove operations run in constant time even in the worst case.

10. 5 points What is the height of the node *de* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes on a longest path starting at v within the subtree of v (not counting the dummy “NONE” nodes that have height 0).

Height of *de*: 2 (Write only a single number.)

Height of the root: 6 (Write only a single number.)



Does the tree satisfy the AVL invariant? Your answer: yes (Write “yes” or “no”.)

Write the names of two nodes,

$X =$ ff $\quad \text{and} \quad Y =$ cu $, \quad \text{such that the following holds:}$

- If the tree violates the AVL invariant, then the violation occurs at node X (caused by the heights of X 's children), and adding a new child to node Y will restore the AVL invariant.
- Otherwise, if the tree satisfies the AVL invariant, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y 's children).
- If there is more than one valid choice for X , choose the one with the alphabetically **largest** name. **After selecting** X , if there is more than one valid choice for Y , choose the one with the alphabetically **smallest** name. (Example: “bb” is alphabetically smaller than “fj”.)

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X . If there are multiple valid choices for X , choose the one with the alphabetically largest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

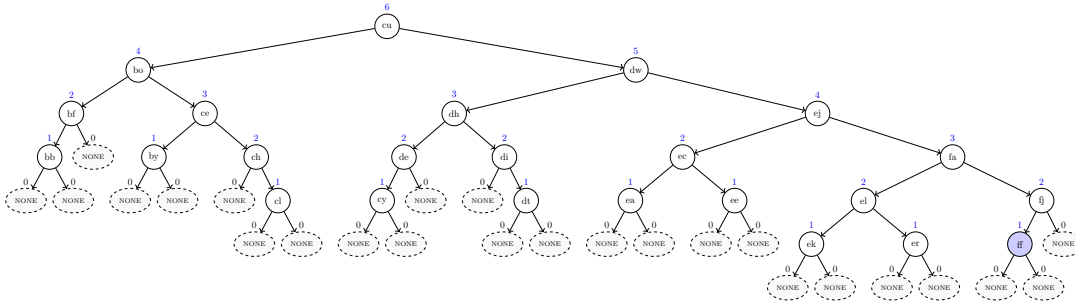
Explanation:

The tree satisfies the AVL invariant. There are seven different candidates for X , where adding a node leads to a violation of the AVL invariant:

bb, cl, cy, dt, ek, er and **ff**.

Recall that out of these different possibilities, the task asks you to choose the one where X is the alphabetically largest node; hence, $X = \text{ff}$.

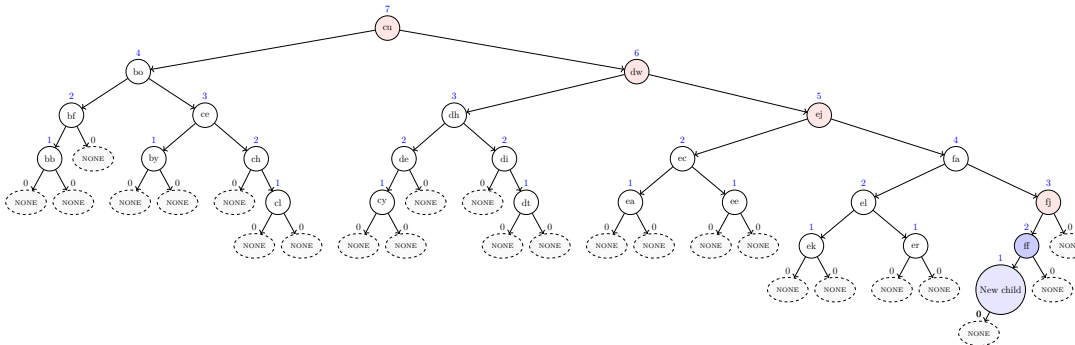
The first tree shows the heights and dummy nodes, as well as the node $X = \text{ff}$ (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the four violating parent nodes in red (whose children differ in height by more than 1):

fj, ej, dw and **cu**.

Among these violating nodes, the task asks you to choose the alphabetically smallest one; hence $Y = \text{cu}$.



For completeness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **bb** → **bf**
- Candidate **cl** → **ch**, **ce** and **bo**
- Candidate **cy** → **de**
- Candidate **dt** → **di**
- Candidate **ek** → **ej**, **dw** and **cu**
- Candidate **er** → **ej**, **dw** and **cu**

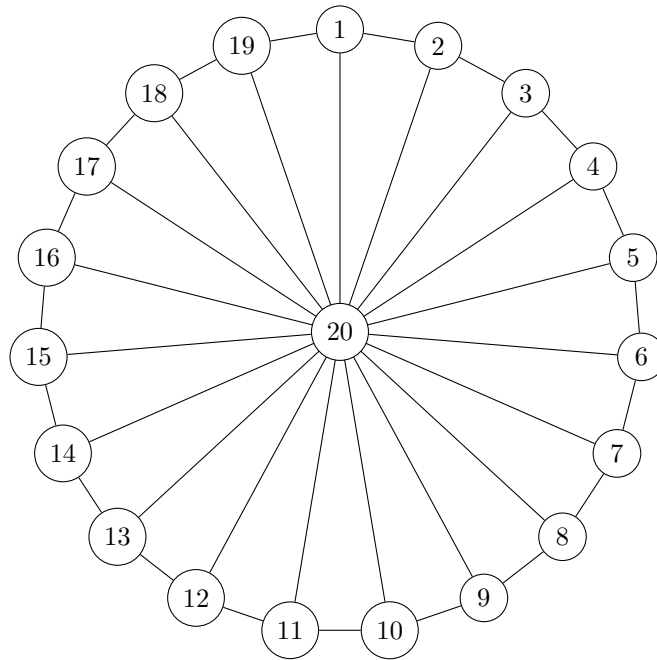
11. 5 points Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to $n=20$ and each edge has the same weight 1. Observe that the vertices 1 to 19 lie on a big cycle and each of them is connected to the center vertex 20 via an edge.

Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 11 iterations.

```

1  $\delta[1..20][1..20]$  = new 2D array filled with  $+\infty$ 
2 for  $i = 1$  to 20 do
3    $\delta[i][i] = 0$ 
4 foreach  $edge(v, w, c)$  do
5    $\delta[v][w] = c$ 
6    $\delta[w][v] = c$ 
7 for  $k = 1$  to 11 do
8   foreach  $pair(v, w)$  do
9      $\delta[v][w] = \min(\delta[v][w], \delta[v][k] + \delta[k][w])$ 
10 return  $\delta$ 

```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 11 times**!

Recall that the weight of every edge is equal to 1.

If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.

$$\delta[18][11] = \underline{\text{INF}} \qquad \delta[20][18] = \underline{1}$$

$$\delta[6][15] = \underline{\text{INF}} \qquad \delta[12][19] = \underline{12}$$

$$\delta[1][6] = \underline{5}$$

12. 4 points Consider the following algorithm.

Input: integer h

$u = 0$

$l = 2$

while $l \leq h$ **do**

$u = u + 4$

$r = 2$

$l = l - r$

$u = u - r$

$l = l + 8$

Someone wrote the following invariant for the loop:

$$B \cdot l = A \cdot u + T$$

with the three constants

- $B = 15$
- $A = 36$
- $T = 24$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, B , A , or T has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? **B** (*Write the name of one of the three constants.*)

The new value for the chosen constant: **12**

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are configuring n Boolean security sensors, and the system's safety requirements are expressed as a complex logical formula over these sensor variables using “and”, “or”, and “not”. You are also given a specific assignment of true/false to each sensor. Does this assignment satisfy the safety formula?

Consider the following decision problem B:

You manage a server with limited disk space. You have a set of files, all of which have exactly the same size s GB, and each file has an importance rating. Can you choose a subset of these files whose total size does not exceed S GB and whose total importance is at least V ?

Mark all the correct sentences and give a short explanation.

- ☒ **A can be reduced to B in polynomial time.**
- ☒ **B can be reduced to A in polynomial time.**
- ☒ **A is in P.**
- ☒ **B is in P.**
- ☐ It is unknown whether A can be reduced to B in polynomial time.
- ☐ It is unknown whether B can be reduced to A in polynomial time.
- ☐ It is unknown whether A is in P.
- ☐ It is unknown whether B is in P.

Solution: Both problems A and B are in P as they can be solved in polynomial time. To solve problem A, we evaluate the given assignment in polynomial time by recursively computing each subformula's value. To solve problem B, since every file has identical size s , you can store at most $k = \lfloor S/s \rfloor$ files; simply sort the files by importance in descending order, sum the top k ratings (in polynomial time), and compare to V .

Consequently, as problems A and B can be solved in polynomial time, they can be reduced in polynomial time to any other problem, thus also to each other.